# PROGRAMMING IN JAVA
# ASSIGNMENT 1 – CAMPUS SMART-PARKING SYSTEM

*Design, Implement and Validate an Object-Oriented Model*

Student Name: D DHANUJA

Register Number: 192311427

Section / Batch: 2023

Submission Date: 17/10/2026

GitHub Repository Link:
https://github.com/DhanujaDhanasekar666/CSA0902-192311427

## 1. Understanding the Question

This assignment asks you to build a small Java program for a campus parking facility. The facility has multiple floors, each with a different number of parking slots. When a vehicle arrives, the program finds a free slot and creates a ticket. When it leaves, the program frees the slot and calculates the parking fee. Different vehicle types have different rates. Electric cars may also pay a charging fee. Students and staff with a valid monthly pass park free; visitors pay the normal fee. The design must use object-oriented programming and be easy to extend when new vehicle types or pricing rules are added.

In simple words: create the classes, write the Java code, test normal and invalid situations, explain the design, and submit the source code in GitHub plus a report with evidence.

## 2. Scenario and Assumptions

- Each floor has a fixed number of slots; floors may have different capacities.
- A null slot means free; a non-null registration number means occupied.
- Registration numbers must be non-empty and unique among currently parked vehicles.
- Each started hour is billed as a full hour. Example: 61 minutes is billed as 2 hours.
- Base hourly rates used in the sample implementation: two-wheeler Rs. 10, car Rs. 30, e-bike Rs. 8, delivery van Rs. 60.
- Electric-car charging costs Rs. 12 per kWh and is added to parking fees.
- A valid monthly pass makes the parking charge zero. Charging, if used, remains a separate charge.
- The assignment mentions one-day passes; the provided sample uses a simple pass flag for demonstration. A production version should replace it with a PassType enum and expiry validation.

## 3. Task-by-Task Answers (Tasks 1–12)

### Task 1 – Understand the problem and identify objects

Candidate classes: ParkingFacility manages floors and slots; Vehicle stores registration details and defines common vehicle behaviour; TwoWheeler, Car, ElectricCar, EBike and DeliveryVan define vehicle-specific rates; ParkingTicket records entry and slot details; PricingRule calculates fees; Chargeable represents vehicles that can charge. Main operations are enter, allocate, exit, release and calculate fee. Rules: no double occupancy, no duplicate active registration, invalid positions are rejected, and fees cannot use negative duration.

## Task 2 – Choose data representation

Use String for registration numbers and ticket IDs, int for floor/slot/kWh, LocalDateTime for entry/exit time, long for duration in minutes, and double for money in this teaching example. Use a jagged String[][] because each floor may have a different slot count. Use a Map keyed by registration number to find active tickets quickly. Avoid integer division errors by using ceiling division: (minutes + 59) / 60.

## Task 3 – Control logic and pseudocode

ALLOCATION: validate registration; reject if already active; for each floor; for each slot; if slot is empty, store registration and return floor/slot; if no slot is empty, return FULL. EXIT: find active ticket; validate exit time; calculate minutes; calculate billable hours using ceiling division; apply pass and vehicle rules; add charging fee if applicable; release the slot; return the total.

## Task 4 – Classes, objects and constructors

Classes are templates and objects are real vehicles/tickets/facilities created from them. Constructors validate and initialise fields. Vehicle has a protected constructor so subclasses must provide a registration number. ParkingTicket captures the vehicle, floor, slot, entry time and pass status. Constructor validation prevents invalid objects from being created.

## Task 5 – Static members and object life cycle

The ticket counter is static because it belongs to the whole class, not one ticket. Every new ticket increments the shared counter. Active tickets are stored while vehicles are parked. When a vehicle exits, its ticket is removed from active maps and the slot is freed; the object can later be garbage-collected if no references remain.

## Task 6 – Inheritance hierarchy

Vehicle is the abstract superclass. TwoWheeler, Car, ElectricCar, DeliveryVan and EBike inherit common registration handling. ElectricCar extends Car and implements Chargeable. This is hierarchical inheritance, with a multilevel path Vehicle → Car → ElectricCar. super(registrationNumber) calls the parent constructor before subclass initialisation.

## Task 7 – Method overriding

Each concrete vehicle overrides ratePerHour() to provide its own rate. @Override lets the compiler check that the method signature is correct. The registration number is final because it should not change after construction. equals() and hashCode() use the registration number so vehicles with the same registration are treated as equal.

## Task 8 – Polymorphism

A Vehicle reference can refer to a TwoWheeler, Car, ElectricCar, DeliveryVan or EBike. Calling ratePerHour() uses dynamic dispatch and executes the real object's overridden method. This avoids long if/else chains for every vehicle type. Overloading can be added for different fee input forms; overriding is used for vehicle-specific behaviour.

## Task 9 – Abstraction and interfaces

Vehicle is abstract because a generic vehicle does not have one universal hourly rate. Chargeable is an interface because only some vehicles support charging. PricingRule is an interface that separates pricing policy from parking operations. This makes it possible to add another pricing policy without rewriting ParkingFacility.

## Task 10 – Packages, access modifiers and encapsulation

For a larger submission, place classes in packages such as parking.model, parking.service and parking.app. Fields are private, and public methods provide controlled access. final protects values that should not change. Constructors and methods validate inputs so calling code cannot put the object into an invalid state. The single-file sample keeps the program easy to compile in the online lab; split classes into packages for the repository if required.

## Task 11 – Implement, test and defend the design

Compile and run the Java file. Test normal parking, full capacity, duplicate registration, invalid floor/slot, zero minutes, a duration at a fee boundary, monthly-pass parking, electric charging and equality. Compare actual output with expected output. Explain that the design uses encapsulation, inheritance, overriding, polymorphism, abstraction and interfaces. One limitation is that this teaching version uses an in-memory map, so data disappears when the program stops.

## Task 12 – Re-evaluate and adapt the design

New categories such as e-bikes and delivery vans should be added as subclasses of Vehicle rather than by rewriting the allocation algorithm. A PricingRule interface allows a new pricing policy to be injected. Different floor capacities are handled by the jagged array. For a more complete implementation, model pass types with an enum, store pass expiry dates, and move delivery-van time rules into a separate pricing policy. The existing tested allocation logic can remain unchanged.

## Pseudocode – Campus Smart-Parking System

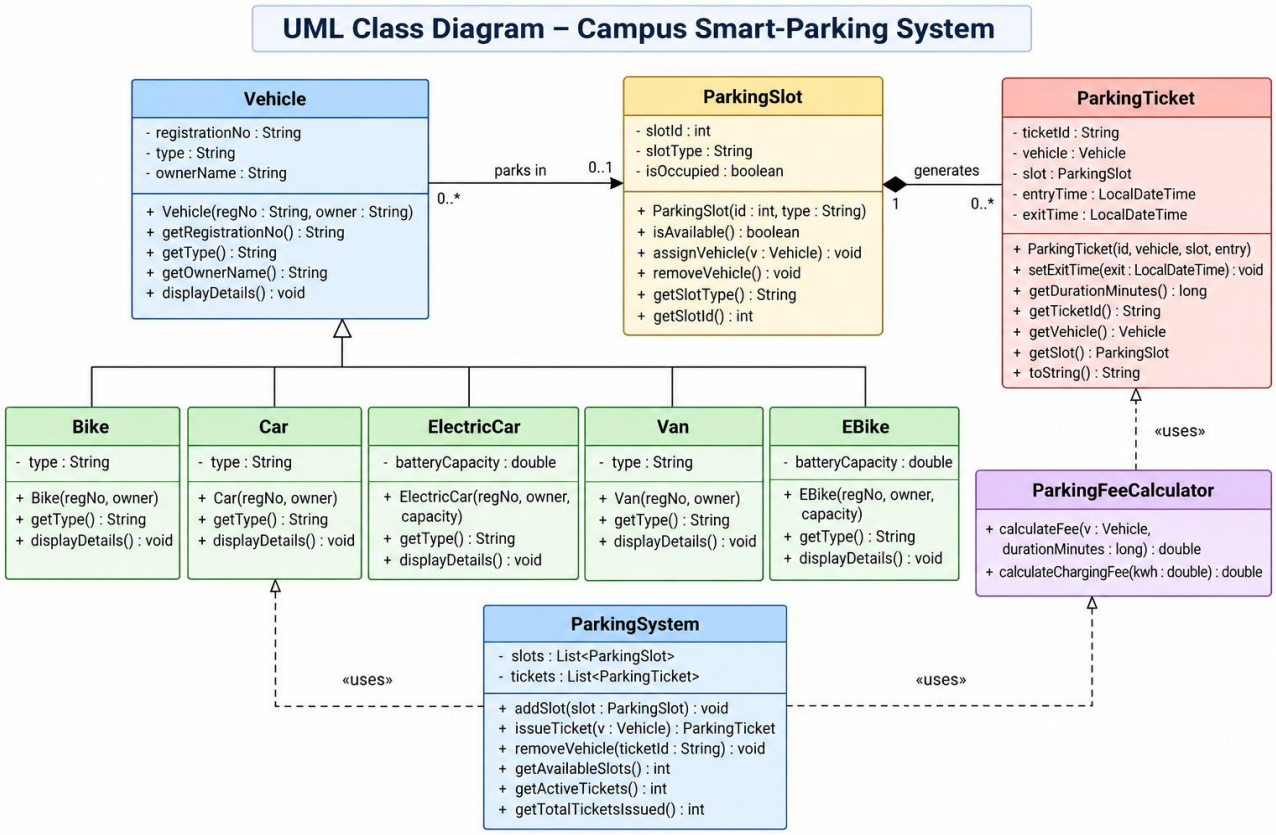
```
ALGORITHM CampusSmartParkingSystem
BEGIN
    INITIALIZE parking floors and slots
    INITIALIZE active tickets
    INITIALIZE ticket counter
    WHILE system is running DO
        DISPLAY "1. Vehicle Entry"
        DISPLAY "2. Vehicle Exit"
        DISPLAY "3. View Available Slots"
        DISPLAY "4. Exit System"
        READ choice
        IF choice = 1 THEN
            READ vehicle registration number
            READ vehicle type
            READ pass type
            IF registration number is empty OR
               vehicle is already parked THEN
                DISPLAY "Invalid or duplicate vehicle"
            ELSE
                SEARCH all floors and slots
```

```
        IF a free slot is found THEN
            ALLOCATE the first free slot
            RECORD vehicle entry time
            GENERATE unique parking ticket
            STORE ticket as active
            DISPLAY ticket details
        ELSE
            DISPLAY "Parking is full"
        END IF
    END IF
ELSE IF choice = 2 THEN
    READ registration number
    FIND active parking ticket
    IF ticket does not exist THEN
        DISPLAY "Invalid or inactive ticket"
    ELSE
        READ exit time
        CALCULATE parking duration in minutes
        IF duration is invalid THEN
            DISPLAY "Invalid parking duration"
        ELSE
            IF a valid monthly or one-day pass
```

```
   covers the parking charge THEN
    parking fee = 0
ELSE
    billable hours = (duration + 59) DIV 60
    SELECT hourly rate based on vehicle type
    IF vehicle is a delivery van AND
       entry time is from 9 AM to 5 PM THEN
        APPLY double car rate
    END IF
    parking fee = billable hours * hourly rate
END IF
IF vehicle supports charging THEN
    READ charging units
    charging fee = charging units * 12
ELSE
    charging fee = 0
END IF
total fee = parking fee + charging fee
DISPLAY parking duration
DISPLAY total fee
FREE the occupied parking slot
REMOVE ticket from active tickets
```

```
                DISPLAY "Vehicle exited successfully"
            END IF
        END IF
    ELSE IF choice = 3 THEN
        COUNT all empty slots
        DISPLAY available slot count
    ELSE IF choice = 4 THEN
        DISPLAY "System closed"
        STOP
    ELSE
        DISPLAY "Invalid choice"
    END IF
  END WHILE
END
```

## 4. UML Class Diagram




## 5. Complexity Analysis

| Operation | Time complexity | Reason |
|---|---|---|
| **Allocate a slot** | O(F × S) worst case | May inspect every slot across F floors with up to S slots each; more precisely O(total slots). |
| **Release by floor/slot** | O(1) | Direct array index access; map removal is O(1) average. |
| **Find ticket by registration** | O(1) average | HashMap lookup by registration number. |
| **Calculate fee** | O(1) | Uses duration, vehicle rate and pass flag. |
| **Space** | O(N + T) | N slot positions plus T active ticket/map. |

## 6. Test Plan and Expected Results

| Test | Input / action | Expected result | Purpose |
|---|---|---|---|
| **T1** | Create facility with capacities {2,1} | Three slots available | Initialisation |
| **T2** | Allocate a valid registration | First free position returned | Normal allocation |
| **T3** | Allocate same active registration again | IllegalArgumentException | Duplicate prevention |
| **T4** | Release invalid floor or slot | false | Boundary validation |
| **T5** | Allocate when all slots are full | null / FULL | Capacity boundary |
| **T6** | Car parks for 130 minutes, no pass | Rs. 90 (3 × Rs. 30) | Fee slab boundary |
| **T7** | Monthly-pass vehicle parks | Parking fee Rs. 0 | Pass rule |
| **T8** | EV uses 10 kWh charging | Rs. 120 charging fee | Interface behaviour |
| **T9** | Construct vehicle with blank registration | IllegalArgumentException | Input validation |
| **T10** | Car and ElectricCar have same registration | equals() returns true | Equality contract |

## 7. Alternative Design Comparison

A one-dimensional array is awkward when floors have different capacities because an index conversion formula needs a fixed number of slots per floor. A jagged array directly represents each floor's own capacity. Searching a jagged array is O(N) in the worst case, where N is the total number of slots. A free-slot stack per floor could make allocation closer to O(1), but it requires extra bookkeeping and careful updates on every entry and exit.

A single method with a switch on vehicle type would become harder to maintain whenever a new vehicle category is introduced. Overriding ratePerHour() lets each subclass supply its rate. The PricingRule interface separates pricing strategy from parking operations, which makes future pricing changes easier to test.

## 8. How to Run the Java Program

1. Save the source as CampusSmartParkingSystem.java.
2. Open a terminal in the folder containing the file.
3. Compile: javac CampusSmartParkingSystem.java
4. Run: java CampusSmartParkingSystem
5. Capture screenshots of the code, successful run, and validation tests.
6. For GitHub, create the required repository, upload the Java file and report, and commit each task progressively.

## 9. Short Viva Answers

**Why use inheritance?** To reuse common vehicle data and behaviour while allowing each type to define its own rate.

**Why is Vehicle abstract?** A generic vehicle has no single correct rate; each concrete type supplies one.

**Why use an interface?** Chargeable and PricingRule describe capabilities/policies without forcing all classes to inherit the same implementation.

**What is polymorphism here?** A Vehicle reference can hold different vehicle objects, and the correct rate method runs at runtime.

**Why use a jagged array?** Each floor can have a different number of slots.

**Why use a HashMap?** It finds an active ticket by registration number in O(1) average time.

**What is encapsulation?** Fields are private and are accessed or changed through controlled methods.

**What is the main limitation?** The example stores data only in memory and does not persist tickets after the program exits.

## 10. Conclusion

The Campus Smart-Parking System models parking slots, vehicles, tickets and pricing rules in Java. The design demonstrates core object-oriented concepts and supports different floor capacities and vehicle categories. Before final submission, run the code in your lab environment, add screenshots of actual outputs, create the required GitHub repository, and replace the blank student details.